



Figure 3: Classification of patterns

```

root@neethu-laptop:~# cat > data
unix shell
322 unix
---unix & linux ----
root@neethu-laptop:~# awk '/unix/{x++}END{print "total count of the pattern : "x}' data
total count of the pattern : 3
  
```

Figure 4: Regular Expression in AWK

regular expressions (regex) — match (~) and doesn't match (!~). Regular expressions must be enclosed in slashes/, as follows:

```

awk '$0 ~ /^[a-d]/' file1 (Identify all lines in a file that
starts with a,b,c or d)
awk '$0 !~ /^[a-d]/' file1 (Identify all lines in a
file that do not start with a,b,c or d)
  
```

An example of an expression is counting the number of occurrences of the pattern 'unix' in the input file 'data'.

Awk supports the following:

Arithmetic operators: +, -, *, /, %, ^.

Relational operators: >, >=, <, <=, ==, != and...

Logical operators: &&, ||, !.

As an example, consider the file *awktest*:

```

1  unix  10  50
2  shell 20  10
3  unix  30  30
4  linux 20  20
• Identify the records with second field, "unix" and
value of third field > 40
awk '$2 == "unix" && $3 > 40 {print}' awktest
1  unix  10  50

• Identify the records where, product of third & fourth
field is greater than 500
awk '$3 * $4 > 500 {print}' awktest

3  unix  30  30
  
```

Hence, no address pattern is entered, and AWK applies action to all the lines in the input file.

The system variables used by AWK are listed below.

- FS: Field separator (default=whitespace)
- RS: Record separator (default=\n)
- NF: Number of fields in the current record
- NR: Number of the current record
- OFS: Output field separator (default=space)
- ORS: Output record separator (default=\n)
- FILENAME: Current file name

There are more than 12 system variables used by AWK. We can define variables (user-defined) also while creating an AWK script.

```

• awk '{OFS=" "; print $1, $2}' marks
  john-85
  andrea-89
• awk '{print NR, $1, $3}' marks
  1 john cse
  2 andrea ece
  
```

Range patterns: These are associated with a range of records, which match a range of consecutive input lines:

Start-pattern, end-pattern{actions}

Range starts with the record that matches the start pattern and ends with the record that matches the end pattern.

Here is an example:

Print 3rd line to 5th line, along with line numbers of the file, marks

```

• awk 'NR==3, NR==5 { print NR, $0}' marks
  
```

Action statements

Expression statements: An expression is evaluated and returns a value, which is either true or false. It consists of any combination of numeric and string constants, variables, operators, functions, and regular expressions.

Here is an example:

```

{$3 = "Hello"}
{sum += ($2+4)}
  
```

Output statements: There are three output actions in AWK: *print*, *printf* and *sprint*.

- *print* writes the specified data to the standard output file. *awk '{print \$1, \$2, \$3}'* file name prints first, second and third columns.
- *printf* uses a format specifier in a 'format-string' that requires arguments of a matching type.
- *string printf* (sprintf) stores the formatted print string as a string.

```

str = sprintf("%2d %-12s %9.2f", $1, $2, $3)
  
```